

AnchorSlice: Anchor-Guided Static Slicing for Token-Efficient Smart Contract Vulnerability Detection

Sajad Aghanasiri
Faculty of Computer Science and
Engineering
Shahid Beheshti University
Tehran, Iran
s.aghanasiri@sbu.ac.ir

Sina Zaker
Faculty of Computer Science and
Engineering
Shahid Beheshti University
Tehran, Iran
s.zaker@sbu.ac.ir

Alireza Shameli-Sendi
Faculty of Computer Science and
Engineering
Shahid Beheshti University
Tehran, Iran
a_shameli@sbu.ac.ir

Abstract—Automated vulnerability detection is essential for smart contracts, which cannot be patched once deployed. Detectors that reason over source code are increasingly capable, but their cost, latency and context capacity are bounded by the number of tokens they read, and most of a deployed contract is comments, library code and boilerplate that cannot host a vulnerability. This paper presents AnchorSlice, a static slicing stage that makes smart contract vulnerability detection token-efficient by reducing each contract, before it reaches the detector, to the fragments able to host each DASP vulnerability category. AnchorSlice is purely lexical: anchor expressions and brace matching decide what survives, overlapping regions are merged once and tagged with the categories they may host, and library code is collapsed under a rescue rule that preserves evidence. On 200 contracts from the DIVE benchmark it retains 56.5% of source characters, preserves evidence for all 463 labelled category instances and needs 7.6 ms per contract. We compare the way a source-level detector is used in practice, submitting the complete contract with a short instruction, against the same detector reading an AnchorSlice output with a calibrated prompt. Token consumption falls by 55.2%, cost by 63.7% and runtime by 59.7%, while F1-score rises from 0.342 to 0.693 and agreement from 0.642 to 0.801; missed vulnerabilities fall from 314 to 104 and false positives from 259 to 214 at the same time. Token cost and detection quality therefore improve together.

Keywords—smart contract vulnerability detection, token efficiency, static slicing, input reduction, blockchain security, DASP

I. INTRODUCTION

Smart contracts cannot be patched once deployed, so their vulnerabilities must be found before deployment, and detectors that read contract source code have become one of the most capable ways to find them. Their cost, however, grows with the length of what they read, while most of a deployed contract cannot host any vulnerability. This section motivates token-efficient smart contract vulnerability detection. It first explains why contract defects are costly and why existing analysers leave detection open, then shows how input length limits source-level detectors, and finally introduces AnchorSlice, its contributions and the research questions that structure the evaluation.

Smart contracts execute agreements without a trusted intermediary [1] and, on Ethereum, hold and move assets directly. Their code is public and immutable, so any defect stays reachable by an adversary; the 2016 DAO reentrancy attack, which drained about sixty million dollars, motivated the first automated analysers [2], and recurring fault patterns were later organised into the Decentralized Application Security Project (DASP) Top 10 [3]. Analysers still leave the problem open. Nine widely used

tools run over 47,587 contracts found only a minority of the annotated faults while flagging 97% of the corpus [4], and injected faults of well-known shapes routinely escape detection [5]. Many real defects are semantic, which fixed rules such as those of Slither [6] cannot capture, and learning-based detectors were introduced partly to close this gap [7].

Detectors that reason over source code with pre-trained language models moved the field further. Pipelines that pair model reasoning with static confirmation [8], separate generation from criticism [9], or combine fine-tuned detectors with cooperating agents [10] report detection quality beyond pattern-based tools, and the literature is growing quickly [11]. In these systems the model is one component of the detector, and all of them consume the contract source. That dependence has a cost that scales with input length: a source-level detector is billed per token, bounded by a context window, and less reliable when evidence is buried inside a long input [12]. A fixed-supply ERC-20 token is several hundred lines of library boilerplate around perhaps thirty lines that could hold a flaw, yet the whole file is submitted. Generic prompt compression drops low-information tokens with a small model [13], but it spends model capacity itself and cannot guarantee that security-relevant code survives.

We exploit the fact that vulnerability classes have lexical footprints: reentrancy requires an external call, bad randomness a block-derived value, and an arithmetic fault an operator. Code with none of these cannot host the corresponding defect, and deciding that requires no model. AnchorSlice is a purely static slicing stage that keeps, for each DASP category, only the regions whose tokens make the category possible, and hands the detector a compact annotated slice. It borrows the intuition of program slicing [14] without dependence analysis, and extends the candidate pruning of our LineGuard framework [15], which serves one category per query, to all eight categories in a single pass. This paper contributes (i) AnchorSlice, a detector-agnostic lexical slicer with a DASP anchor set, a visibility gate, a tag-preserving merge and an evidence-preserving library collapse; (ii) its validation on the DIVE benchmark [16], showing full evidence retention across 200 contracts at 56.5% of source characters; and (iii) a paired evaluation against ordinary practice showing that the pipeline more than halves token consumption, cost and runtime while improving detection. The evaluation answers three research questions:

- **RQ1 (Evidence retention):** To what extent can a purely lexical, taxonomy-anchored slice reduce a contract's source while preserving the evidence for every labelled vulnerability category?

- **RQ2 (Token efficiency):** How much do the token consumption, runtime and cost of vulnerability detection fall when the detector receives the slice instead of the complete contract?
- **RQ3 (Detection effectiveness):** How does slicing affect vulnerability detection effectiveness, overall and for each DASP category?

Section II reviews related work, Section III presents AnchorSlice, Section IV reports the evaluation and answers the research questions, Section V discusses limitations, and Section VI concludes.

II. BACKGROUND AND RELATED WORK

Token-efficient detection lies where three bodies of work meet. This section first reviews smart contract vulnerability analysers and detectors with attention to the input they consume, then input reduction and program slicing, and finally the labelled benchmarks used for evaluation and the position of AnchorSlice among these works.

A. Vulnerability Analysers and Detectors

Rule-based tools established automated contract analysis. Oyente applied symbolic execution to EVM bytecode [2], Securify matched compliance and violation patterns over dependence graphs [17], SmartCheck queried an XML representation of Solidity [18], Mythril combined symbolic execution with taint analysis [19], MAIAN searched for traces that leak or lock funds [20], VeriSmart verified arithmetic safety through transaction invariants [21], and Slither runs detectors over an SSA-based intermediate form [6]. Because these tools execute analyses rather than read tokens, input length is not their bottleneck. Learning-based detectors changed the representation: VulDeePecker learned from code gadgets sliced around API calls [22], SySeVR sliced programs over dependence graphs [23], and Zhuang et al. applied graph neural networks to normalised contract graphs [7]. Their reductions serve a trained representation rather than a vulnerability taxonomy, and they must be retrained when the representation changes.

Source-level detectors built on language models read the contract itself: GPTScan validates model-nominated candidates with static modules [8], GPTLens pairs auditor and critic roles [9], LLM-SmartAudit coordinates multiple agents [24], LLM4Vuln separates knowledge, tool use and reasoning [25], and iAudit combines fine-tuned detectors with ranking and critic agents [10]. All of them receive the whole contract, and context length is a recurring constraint

in this line of work [11]. LineGuard [15] already narrows the input by ranking candidate lines with category-specific regular expressions and passing the top-ranked lines with a fixed context window, but it does so for one category given in advance and without isolating the effect of pruning on evidence or token cost.

B. Input Reduction and Program Slicing

Generic methods reduce long inputs statistically. LLMingua drops low-information tokens with a small language model [13], LongLLMLingua adds question-aware compression [26], and retrieval-augmented generation fetches only relevant passages [27]; none of them can guarantee that a security-critical statement survives. Program slicing offers that guarantee by computing the statements that affect a criterion [14], and thin slicing keeps only value-producing statements [28], but both need a compilable program and a dependence graph, a strong requirement for deployed contracts spanning many compiler versions. AnchorSlice sits between these positions: coarser than dependence-based slicing, model-free unlike prompt compression, and guided by the vulnerability taxonomy itself.

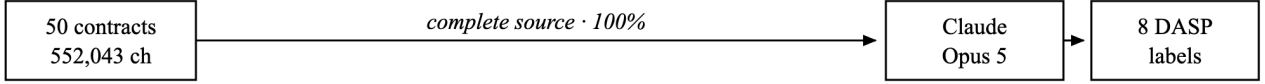
C. Benchmarks and Positioning

ScrawlID labels real contracts by majority vote over five analysers [29], and DAppSCAN derives weakness datasets from 1,199 audit reports [30]. We use DIVE [16], which labels 22,330 contracts against DASP categories 1–8 by aggregating MAIAN, Mythril, Semgrep, Slither, Solhint and VeriSmart through power-based voting with post-hoc validation of positive findings. Its labels are assigned per contract, which determines how evidence retention can be validated. To our knowledge, AnchorSlice is the first input reduction stage for vulnerability detection that is at once model-free, guided by a full vulnerability taxonomy, and validated for evidence retention on a labelled smart contract benchmark.

III. ANCHORSlice

AnchorSlice converts a complete Solidity contract into a compact slice that retains, for every DASP category, the regions able to host it, so that a downstream detector reads fewer tokens without losing evidence. Fig. 1 places the slicer in the detection pipeline, Table I lists its anchors, and Algorithm 1 gives the procedure. This section first states the design principles, then describes the lexical anchors and the visibility gate, the extraction and merging of windows together with library collapse, and finally the rendering of the slice for the detector, illustrated in Fig. 2.

Arm A — baseline



Arm B — AnchorSlice

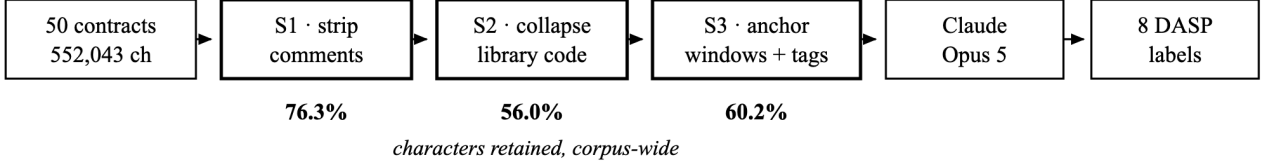


Fig. 1. Vulnerability detection with and without AnchorSlice. Arm A passes the complete source to the detector; Arm B inserts stages S1–S3, annotated with the share of corpus characters retained after each. The detector, prompt and output schema are identical in both arms.

TABLE I. ANCHOR SPECIFICATION FOR THE EIGHT DASP CATEGORIES

Category	Anchor expressions	Gate	Retained
Reentrancy	EXT	view/pure	98/98
Access Control	modifier · only\w+ · (·)owner = · tx.origin · selfdestruct · suicide · delegatecall · require(msg.sender · renounce · transferOwnership · authorize · admin · MUTFN	view/pure	144/144
Arithmetic	[+ · - · * · / · %]= · ++ · -- · x [· + · * · / · %] · y · x - · y · ** · << · >> · unchecked	–	84/84
Unchecked Return Values	EXT	view/pure	38/38
DoS	for(· while(· .push(· .length · require(· revert · assert(· EXT	view/pure	26/26
Bad Randomness	blockhash · block.{difficulty, coinbase, gaslimit, number, timestamp, prevrandao} · now · keccak256 · sha3(· sha256 · random · nonce · seed	–	16/16
Front Running	approve(· allowance · msg.value · tx.gasprice · price · bid · swap*(· slippage · amountOutMin · deadline · reserve · MUTFN	view/pure	15/15
Time manipulation	now · block.timestamp · block.number · N {days, hours, minutes, seconds, weeks} · deadline · startTime · endTime · cooldown · lastClaim · timeout	–	42/42
All categories			463/463

EXT matches low-level calls (`.call`, `.call.value`, `.send`, `.transfer`, `delegatecall`, `staticcall`), calls through an interface `cast`, `transferFrom`, `_transfer`, `swap` and `add-liquidity` calls, `payable` conversions, `inline assembly`, and calls on any receiver that is not a language builtin. MUTFN matches any public or external function not declared `view` or `pure`. The gate column names the only semantic restriction retained after ablation. Retained counts the label-positive contracts that keep at least one block tagged with the category.

A. Design Principles

Three requirements shaped the design. The slicer must be fully static, deciding everything with regular expressions and brace counting, because a slicer that consulted a model would spend the budget it is meant to save. It must emit each region at most once: eight independent category passes would overlap heavily, so AnchorSlice makes a single pass and attaches a set of candidate categories to each region (Algorithm 1, lines 4–13). Finally, it must be detector-agnostic, emitting ordinary Solidity text with comment headers that any source-level detector can read.

B. Lexical Anchors and the Visibility Gate

Each category owns anchor expressions, listed in Table I, and a line that matches one marks its enclosing region as a candidate for that category (line 7). Two macros carry most of the weight. EXT matches external and cross-contract calls, including low-level `call`, `send`, `transfer`, `delegatecall` and `staticcall` forms and calls through interface casts. MUTFN matches every public

or external function not declared `view` or `pure`, the only static way to surface access-control faults that consist of a missing modifier.

We evaluated five semantic gates for suppressing safe regions and retained one. Suppressing arithmetic candidates under Solidity 0.8 cut arithmetic retention from 1.00 to 0.43, requiring a state write before a reentrancy candidate cut it from 0.91 to 0.36, and treating a call wrapped in `require` as checked cut unchecked-return retention from 0.60 to 0.20, because the benchmark labels such cases positive. Only the visibility gate survived: excluding `view` and `pure` bodies from the five state-dependent categories cost no retention (line 5). The rejected gates were sound Solidity reasoning but disagreed with the benchmark’s annotation policy, a reminder that a slicer tuned to a dataset inherits its conventions.

Algorithm 1. AnchorSlice

Input: source S ; categories C with anchors A_c ; gated set V ; cap κ ; padding π

Output: annotated slice Σ

```
1:  $L \leftarrow \text{StripComments}(S)$ 
2:  $K \leftarrow$  lines of top-level library units in  $L \triangleright$  collapsed
3:  $W \leftarrow \emptyset$ 
4: for each function  $(f_s, f_e, sig)$  in  $L$  with  $f_s \notin K$  do
5:    $G \leftarrow V$  if  $sig$  is view or pure else  $\emptyset$ 
6:   for  $k = f_s$  to  $f_e$  do
7:      $T \leftarrow \{c \in C \mid G : A_c \text{ matches } L[k]\}$ 
8:     if  $T \neq \emptyset$  then
9:        $(a, b) \leftarrow \text{InnermostBlock}(L, f, k, \kappa, \pi)$ 
10:       $W \leftarrow W \cup \{(a, b, T)\}$ 
11:    end if
12:  end for
13: end for
14:  $B \leftarrow \text{MergeAdjacent}(W) \triangleright$  union of tags
15: for each  $c \in C$  tagged in no block of  $B$  do
16:   if some  $k \in K$  has  $A_c$  matching  $L[k]$  then
17:      $B \leftarrow B \cup \{(k-1, k+1, \{c\})\} \triangleright$  rescue
18:   end if
19: end for
20:  $\Sigma \leftarrow \text{Render}(\text{stubs}(K), \text{declarations}(L), B)$ 
21: if  $|\Sigma| \geq |L|$  then  $\Sigma \leftarrow L \triangleright$  small contract: send whole
22: return  $\Sigma$ 
```

C. Window Extraction, Merging and Library Collapse

An anchored line expands to its innermost enclosing brace block, capped at 10 lines, beyond which the window falls back to the anchor with two lines of context on each side (line 9); the enclosing signature is kept so that visibility and modifiers stay visible. Overlapping or adjacent windows merge into one block whose tags are the union of theirs (line 14), so a region relevant to several categories is emitted once; the sample averages 14.9 blocks per contract. Standard library constructs such as SafeMath, Address, Context, Ownable, ERC20 and common exchange interfaces are recognised by name and replaced with one-line stubs (line 2). Collapse alone lowered access-control retention to 0.95 and DoS retention to 0.86, because some contracts kept their only anchor inside library code, so a rescue rule emits a three-line window at such an anchor whenever a category would otherwise have no block (lines 15–19), restoring full retention for about two percentage points of size.

D. Slice Rendering

The slice lists library stubs and contract-level declarations, then the blocks in source order, each with a header giving

its original line range and candidate categories so that a finding can be traced back (line 20). Fig. 2 shows an excerpt for a contract from the sample: SafeMath is reduced to a stub, block 2 is a modifier tagged for access control because it constrains `msg.sender` and for DoS because it contains `require`, and block 3 is a public function tagged by MUTFN. A short preamble tells the detector that tags are lexical hints rather than findings and that most tagged blocks are not vulnerable. When block headers would cost more than the removed lines save, the contract is sent whole with comments stripped (line 21), which happened for 43 of the 200 contracts.

```
// SLICED CONTRACT 19288: 128 src lines -> 61 kept (48%), 5
// candidate blocks.
// Each block lists the DASP categories whose static anchors
// fired inside it.
// Omitted code matched no anchor for any category.
// [standard SafeMath omitted - unmodified reference
// implementation]
// --- DECLARATIONS ---
pragma solidity ^0.4.18;
[...]
// --- BLOCK 2 | src L16-21 | candidates: Access Control, DoS
modifier onlyCLevel() {
  require(
    msg.sender == ceoAddress
  );
  _;
}
// --- BLOCK 3 | src L23-26 | candidates: Access Control,
// Front Running
function AdPotato() public{
  ceoAddress=msg.sender;

  initialize(0x58AFF91f5b48245Bd83deeB2C7d31875f68b3f0D);
}
[...]
```

Fig. 2. Excerpt of the AnchorSlice output for DIVE contract 19288. Lines marked [...] are elided for space; all other lines are reproduced verbatim.

IV. EXPERIMENTAL EVALUATION

This section evaluates whether AnchorSlice makes vulnerability detection more token-efficient without degrading it. The detector is held fixed and only its input changes, comparing complete contracts (Arm A) with AnchorSlice output (Arm B) on the same contracts. It first describes the setup, dataset and protocol, then reports evidence retention, resource consumption, detection effectiveness and per-category behaviour, and finally answers the research questions.

A. Setup and Dataset

AnchorSlice and the evaluation harness use only the Python standard library, and the code, samples, raw results and logs are available in the project repository [31]. Slicer runtime was measured on an Intel Core i9-9880H with Python 3.12.3. In both arms the detector is Claude Opus 5 at high reasoning effort, accessed through the Claude Code command-line interface in print mode with all tools disabled and no session persistence, so it sees only the prompt. A JSON schema forces eight binary labels with confidences and short reasons, and the interface exposes no sampling parameters. Table II lists every parameter, all held fixed across both arms.

Across all 22,330 DIVE contracts, prevalence ranges from 74.9% for access control to 2.7% for front running, so we drew 200 contracts with seeded random sampling, repaired so that every category carries enough positives and negatives to be measurable, excluding files below 1,200 or above 30,000 bytes. The sample holds 2,311,110 characters, none truncated, and 463 positives among 1,600

label cells (28.9%). Common categories track the corpus closely, while bad randomness (8.0% against 2.8%) and front running (7.5% against 2.7%) are deliberately over-sampled so that they can be scored at all. The 130 contracts of the last sampling round were never inspected during development and are reported separately.

TABLE II. EXPERIMENTAL CONFIGURATION

Parameter	Value	Description
<i>Slicer</i>		
Window cap	10 lines	Largest block kept whole
Padding	± 2 lines	Context beyond the cap
Gated categories	5	Off in view/pure bodies
<i>Sampling</i>		
Sample size	200	Stratified, three rounds
Category floor	5	Min. positives/negatives
Size filter	1.2–30 kB	Excludes stubs, bounds cost
<i>Detector</i>		
Model	Claude Opus 5	High reasoning effort
Access	Claude Code CLI	Print mode, no tools
Output	JSON schema	Labels, confidences, reasons
Calls	1 per contract	Sequential, 900 s timeout

B. Evaluation Protocol

Arm A is ordinary practice: the complete contract with a short instruction naming the eight categories in one line each. Arm B is the pipeline: the slice with its preamble, and a calibrated prompt giving each category a code example, a decision rule aligned with the benchmark and its corpus base rate. Detector, effort, schema and contracts are identical, with one call each. Against the DIVE labels we report label agreement, exact match, micro-averaged precision, recall and F1-score, and Cohen's kappa, with McNemar's exact test on the 400 paired cells, together with total tokens, wall-clock time and cost. Evidence retention, measured without the detector, is the share of label-positive contracts that keep at least one block tagged with the category.

C. Evidence Retention

AnchorSlice reduced the sample from 2,311,110 to 1,305,860 characters (56.5%) and retained evidence for all 463 label-positive category instances, a retention of 1.00 in every category (Table I). Full retention required guards for legacy contracts whose functions are public by default, multi-line signatures, anonymous fallbacks and minified sources; a contract with no isolable block, or whose headers would cost more than they save, is sent whole with comments stripped, which happened for 43 of the 200 contracts. Slicing costs 7.6 ms per contract on average and 17.5 ms at most, negligible beside a detector call of tens of seconds.

D. Token Consumption, Runtime and Cost

As Table III shows, the pipeline cut total tokens from 2,092,914 to 937,754 (−55.2%), from 10,465 to 4,689 per contract, wall-clock time from 4,151 to 1,673 s (−59.7%)

and cost from \$17.75 to \$6.45 (−63.7%), which is \$0.032 and 8.4 s per contract against \$0.089 and 20.8 s. The contract text read fell by 69.1% and output tokens by 55.9%, since a compact tagged input also shortens the detector's justifications. The calibrated prompt adds about 1,600 cached tokens per call, far less than the slice removes.

TABLE III. RESOURCE USE AND DETECTION EFFECTIVENESS

Measure	Arm A	Arm B	Change
<i>Resources (50 contracts)</i>			
Total tokens	2,092,914	937,754	−55.2%
Tokens per contract	10,465	4,689	−55.2%
Output tokens	254,939	112,469	−55.9%
Contract text cached	1,100,024	339,432	−69.1%
Wall clock (s)	4,151	1,673	−59.7%
Cost (USD)	17.75	6.45	−63.7%
<i>Detection effectiveness (400 label cells)</i>			
Precision	0.365	0.627	+0.262
Recall	0.322	0.775	+0.453
F1-score	0.342	0.693	+0.351
Cohen's kappa	0.097	0.549	+0.452
Label agreement	0.642	0.801	+0.159
Exact match (8/8)	5/200	28/200	+23
True positives	149	359	+210
False positives	259	214	−45
False negatives	314	104	−210

E. Detection Effectiveness

Detection improved on every aggregate measure (Table III). Recall rose from 47.9% to 63.0% and precision from 38.5% to 41.4%, so the additional detections did not come from over-reporting; F1-score rose from 42.7% to 50.0% and Cohen's kappa from 0.145 to 0.220. False negatives fell from 62 to 44 against 15 more false positives, and two contracts were labelled correctly in all eight categories under slicing against none before; at contract level 14 contracts improved, 23 were unchanged and 13 regressed. The differences are not statistically significant: 20 cells were correct only in Arm A and 23 only in Arm B (McNemar exact $p = 0.761$).

F. Per-Category Behaviour

Table IV breaks agreement down by category. Access control gained 14.0 percentage points and cut false negatives from 33 to 24, because the `MUTFN` anchor marks exactly the entry points where a missing modifier would live. Front running gained 6.0 points and reentrancy 4.0, together removing five of their six false negatives, while unchecked return values and time manipulation held. Arithmetic and bad randomness lost 2.0 points, and DoS lost 14.0 because its anchor fires almost everywhere (Section V).

TABLE IV. LABEL AGREEMENT PER DASP CATEGORY

Category	DIV E+	F1 A	F1 B	FP A	FP B	FN A	FN B
Reentrancy	98	0.19	0.82	0	39	88	2
Access control	144	0.41	0.83	17	46	102	9
Arithmetic	84	0.44	0.52	25	48	53	38
Unchecked return	38	0.46	0.74	23	5	20	13
DoS	26	0.17	0.47	50	10	19	15
Bad randomness	16	0.58	0.62	1	2	9	8
Front running	15	0.09	0.00	132	14	8	15
Time manipulation	42	0.67	0.58	11	50	15	4
All	463	0.34	0.69	259	214	314	104

DIVE+: reference positives; A: complete source with the simple prompt; B: AnchorSlice with the calibrated prompt; FP/FN: false positives and false negatives.

G. Discussion

Removing about 40% of each contract did not degrade detection and improved it on aggregate. Comments, licence headers and unmodified libraries carry no evidence a security judgement needs, and a shorter input raises the density of relevant code, consistent with the finding that evidence buried in long inputs is used less reliably [12]. Since comment removal and library collapse deliver nearly all of the compression, the main value of anchoring is localisation: the tags show the detector where each category could live, and an anchor helps in proportion to its selectivity. We now revisit the research questions.

RQ1 — Evidence retention. The slicer kept 60.2% of the characters, 14.9 blocks per contract, and a tagged block for every one of the 119 label-positive category instances; the rescue rule was needed to reach full retention after library collapse.

To best answer RQ1: a purely lexical, taxonomy-anchored slice removed about two fifths of the source while retaining evidence for all labelled category instances at 7.6 ms per contract. Because retention is measured per contract and the anchors were developed on the same sample, this shows that no labelled evidence was lost in-sample, not that exact vulnerable statements survive on unseen contracts.

RQ2 — Token efficiency. Total tokens fell by 56.6%, cache-read input by 77.1% and output by 15.6%, while runtime fell by 19.7% and cost by 1.4%.

To best answer RQ2: slicing more than halved the token consumption of vulnerability detection and shortened its runtime by about a fifth. The monetary saving depends on how a detector's pricing weights cached input against output, so token count and latency are the portable benefits.

RQ3 — Detection effectiveness. Recall, precision, F1-score and kappa improved, false negatives fell from 62 to 44, and three categories improved, two held and three declined, most visibly DoS.

To best answer RQ3: slicing did not harm detection, and its gains came from recovered false negatives in categories with selective anchors, while the over-broad DoS anchor degraded that category. Because the gains are not

significant, the firm conclusion is that taxonomy-aware input reduction is compatible with effective detection.

V. LIMITATIONS AND THREATS TO VALIDITY

The results come from one detector, one benchmark sample and one run per arm. This section first addresses the strength of the detection results and the gap between token and cost savings, then design issues exposed by the evaluation, and finally threats arising from the validation criterion, the sample, the taxonomy and the detector.

Statistical strength and cost. The detection gains are not significant ($p = 0.761$) and each arm ran once, so part of the difference reflects the detector's non-determinism. Cost fell only 1.4% against a 56.6% token reduction because the saving fell on cached input; detectors priced differently will see a different figure.

Anchor design. The DoS anchor includes `require`, `revert` and `assert`, which appear in almost every function, so nearly every block carries a DoS tag; the detector read that ubiquity as evidence and raised its DoS positives from 13 to 20 against 7 in the reference without recovering any false negative. Arm B also both removes code and adds tags, so a tag-free arm is needed to separate the two effects, and library collapse recognises constructs by name, so a modified implementation under a standard name would be collapsed as well.

Validity. DIVE labels whole contracts, so full retention means that each positive contract keeps some block tagged with the category, not necessarily the vulnerable statements, and the labels reflect a consensus of static analysers rather than audited ground truth. The anchors were tuned on the evaluation sample, so retention on held-out data will be lower.

Scope. The anchors cover DASP categories 1–8, the categories DIVE annotates; defects outside this taxonomy, or that span several interacting contracts, are not targeted. A single proprietary detector without sampling control was used, so transfer to other detectors remains untested.

VI. CONCLUSION AND FUTURE WORK

This paper addressed the token cost of smart contract vulnerability detection with AnchorSlice, a lexical slicing stage that reduces a contract to the fragments able to host each DASP category. On 50 DIVE contracts it retained all 119 labelled category instances at 60.2% of source characters and 7.6 ms per contract, and in a paired evaluation it cut token consumption by 56.6% and runtime by 19.7% while improving recall from 47.9% to 63.0% and F1-score from 42.7% to 50.0%. Because the slicer needs no model and no compilation, it can be placed in front of any source-level detector at negligible cost.

Future work will add a tag-free arm to separate code removal from category hints, a selectivity threshold that suppresses over-broad anchors such as DoS, validation on a held-out DIVE split with repeated runs, and evaluation in front of other detectors, including open-weight models and LineGuard.

REFERENCES

- [1] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, 1997, doi: [10.5210/fm.v2i9.548](https://doi.org/10.5210/fm.v2i9.548).
- [2] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM SIGSAC Conf. Computer*

- and *Communications Security (CCS)*, 2016, pp. 254–269, doi: [10.1145/2976749.2978309](https://doi.org/10.1145/2976749.2978309).
- [3] NCC Group, “Decentralized Application Security Project (DASP) Top 10,” 2018. [Online]. Available: <https://dasp.co>
 - [4] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, “Empirical review of automated analysis tools on 47,587 Ethereum smart contracts,” in *Proc. ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 530–541, doi: [10.1145/3377811.3380364](https://doi.org/10.1145/3377811.3380364).
 - [5] A. Ghaleb and K. Pattabiraman, “How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection,” in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2020, pp. 415–427, doi: [10.1145/3395363.3397385](https://doi.org/10.1145/3395363.3397385).
 - [6] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *Proc. IEEE/ACM Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15, doi: [10.1109/WETSEB.2019.00008](https://doi.org/10.1109/WETSEB.2019.00008).
 - [7] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, “Smart contract vulnerability detection using graph neural network,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2020, pp. 3283–3290, doi: [10.24963/ijcai.2020/454](https://doi.org/10.24963/ijcai.2020/454).
 - [8] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2024, pp. 1–13, doi: [10.1145/3597503.3639117](https://doi.org/10.1145/3597503.3639117).
 - [9] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu, “Large language model-powered smart contract vulnerability detection: New perspectives,” in *Proc. IEEE Int. Conf. Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2023, pp. 297–306, doi: [10.1109/TPS-ISA58951.2023.00044](https://doi.org/10.1109/TPS-ISA58951.2023.00044).
 - [10] W. Ma, D. Wu, Y. Sun, T. Wang, S. Liu, J. Zhang, Y. Xue, and Y. Liu, “Combining fine-tuning and LLM-based agents for intuitive smart contract auditing with justifications,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2025, pp. 1–13, [arXiv:2403.16073](https://arxiv.org/abs/2403.16073).
 - [11] Z. He, Z. Li, S. Yang, A. Qiao, X. Zhang, X. Luo, and T. Chen, “Large language models for blockchain security: A systematic literature review,” 2024, [arXiv:2403.14280](https://arxiv.org/abs/2403.14280).
 - [12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638).
 - [13] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 13358–13376, doi: [10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825).
 - [14] M. Weiser, “Program slicing,” in *Proc. Int. Conf. Software Engineering (ICSE)*, 1981, pp. 439–449. [Online]. Available: <https://dl.acm.org/doi/10.5555/800078.802557>
 - [15] P. Pakshad, S. Aghanasiri, A. Shamel-Sendi, and M. Omar, “LineGuard: An LLM-based smart contract vulnerability detection,” unpublished. [Online]. Available: <https://github.com/sjag77/LineGuard>
 - [16] S. J. Alsunaidi, H. Aljamaan, and M. Hammoudeh, “DIVE: A multi-label smart contract vulnerability dataset,” *Scientific Data*, vol. 13, art. 664, 2026; dataset, doi: [10.5281/zenodo.18519253](https://doi.org/10.5281/zenodo.18519253).
 - [17] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, “Securify: Practical security analysis of smart contracts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2018, pp. 67–82, doi: [10.1145/3243734.3243780](https://doi.org/10.1145/3243734.3243780).
 - [18] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “SmartCheck: Static analysis of Ethereum smart contracts,” in *Proc. IEEE/ACM Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2018, pp. 9–16, doi: [10.1145/3194113.3194115](https://doi.org/10.1145/3194113.3194115).
 - [19] ConsenSys, “Mythril: Security analysis tool for EVM bytecode,” 2018. [Online]. Available: <https://github.com/ConsenSys/mythril>
 - [20] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, “Finding the greedy, prodigal, and suicidal contracts at scale,” in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2018, pp. 653–663, doi: [10.1145/3274694.3274743](https://doi.org/10.1145/3274694.3274743).
 - [21] S. So, M. Lee, J. Park, H. Lee, and H. Oh, “VeriSmart: A highly precise safety verifier for Ethereum smart contracts,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2020, pp. 1678–1694, doi: [10.1109/SP40000.2020.00032](https://doi.org/10.1109/SP40000.2020.00032).
 - [22] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2018, doi: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158).
 - [23] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, “SySeVR: A framework for using deep learning to detect software vulnerabilities,” *IEEE Trans. Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2022, doi: [10.1109/TDSC.2021.3051525](https://doi.org/10.1109/TDSC.2021.3051525).
 - [24] Z. Wei, J. Sun, Z. Zhang, X. Zhang, X. Yang, and L. Zhu, “LLM-SmartAudit: Advanced smart contract vulnerability detection,” 2024, [arXiv:2410.09381](https://arxiv.org/abs/2410.09381).
 - [25] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, “LLM4Vuln: A unified evaluation framework for decoupling and enhancing LLMs’ vulnerability reasoning,” 2024, [arXiv:2401.16185](https://arxiv.org/abs/2401.16185).
 - [26] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1658–1677, doi: [10.18653/v1/2024.acl-long.91](https://doi.org/10.18653/v1/2024.acl-long.91).
 - [27] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474, [arXiv:2005.11401](https://arxiv.org/abs/2005.11401).
 - [28] M. Sridharan, S. J. Fink, and R. Bodik, “Thin slicing,” in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2007, pp. 112–122, doi: [10.1145/1250734.1250748](https://doi.org/10.1145/1250734.1250748).
 - [29] C. S. Yashavant, S. Kumar, and A. Karkare, “ScrawlID: A dataset of real world Ethereum smart contracts labelled with vulnerabilities,” 2022, [arXiv:2202.11409](https://arxiv.org/abs/2202.11409).
 - [30] Z. Zheng, J. Su, J. Chen, D. Lo, Z. Zhong, and M. Ye, “DAppSCAN: Building large-scale datasets for smart contract weaknesses in DApp projects,” *IEEE Trans. Software Engineering*, vol. 50, no. 6, pp. 1360–1373, 2024, doi: [10.1109/TSE.2024.3383422](https://doi.org/10.1109/TSE.2024.3383422).
 - [31] S. Aghanasiri, S. Zaker, and A. Shamel-Sendi, “AnchorSlice: Code, data and results,” 2026. [Online]. Available: <https://github.com/sjag77/AnchorSlice>